

Testril demos: the decision brief

Proposal · 3 October 2026 · Two-page review

The story to own

Buy the answer. Inspect the evidence. Earn when the data gets reused. When function creation ships: **if the metric doesn't exist, create it.**

Start with agent/application developers solving a protocol team's concrete problem. Make the engineer look good: they deliver a useful answer, defend the number, and put it in a product. Full replacement of Dune, Allium, and Nansen is the direction; the first win is replacing one valuable workflow.

Lead with useful work in fractions of a cent. Competitors are adding machine payments to existing API products; Testril starts with tiny purchases, paid materialization, and reuse. Familiar billing can sit above that foundation. Nansen advertises 1¢/5¢ calls; Allium lists 1¢ SQL submission and 2¢/3¢ data calls. The opportunity is economically practical fine-grained agent work. [Pricing evidence](#).

Before function creation: release a connected series

Opening: One Cent, Many Decisions. Use existing transfer volume to investigate token activity through selective narrow reads. Show each result, next decision, receipt, and remaining allowance. At current rates, fifty 100-block cached reads total **\$0.006: 0.6 cents**. Rehearse a useful sequence; disclose preparation separately. This demonstrates tiny purchases before new functions ship.

1. Follow the Money — the flagship. “Where did this treasury's USDC go?” Give the agent a wallet, window, and spending limit. A flow diagram explains counterparties and net movement. Separate internal transfers using the team's supplied addresses. Click a number to inspect its evidence; export the case file and runnable integration. Start with one token on one chain. Required addition: **transfer edges by block**, with exact amounts and counterparties; verify exact balance snapshots for reconciliation.

2. The Second Customer Pays You. A different app buys a read of the same prepared data. Show the original materializer's actual earned balance increasing, beside what both parties paid. Reuse the claim dashboard for an eventual payout. Show real customer roles; label operator-funded demonstration traffic. This proves the earning mechanism, not profitable demand.

3. Show Me the Evidence. Reproduce the flagship's calculation from pinned sources; deliberately alter a displayed amount and expose the mismatch. This is an evidence/export mode of the flagship, not another application. A source-block digest alone does not prove the arithmetic.

Later episodes: Did Liquidity Leave? for DeFi customers; **Ship the Endpoint** for conventional REST. These need pool functions or a gateway, respectively.

After function creation: make the missing metric the reveal

4. The Metric That Didn't Exist. A vault team invents a deposit-cohort rule. Show a catalog miss, a precise definition, newly generated code, meaningful validation, a quote, and the answer. A fresh client then reuses the new function. Selecting an existing function or changing its parameters does not qualify.

5. Did the Campaign Stick? Create and revise a repeat-depositor definition, showing observation cutoffs and traceable results. This needs stateful computation and timestamps as well as generation.

6. Ask → Build → Earn. Combine creation with a second customer's paid reuse. This is the capstone once both halves work. Today's earnings attach to materialization; code-author royalties are a separate decision.

Make the wallet useful; make the first encounter easy

Present it as a programmable purchasing identity: a visible allowance, scoped authorization, receipts, and eligible earnings. Offer **watch**, **sponsored live trial**, and **bring a wallet**. A visitor can experience value before funding a wallet. The sponsor remains the payer and entitlement holder unless a different arrangement is explicitly implemented.

Enforce budgets in the signer/client, outside the LLM. Handle batch escrow and vouchers correctly; show deposited funds separately from charges. Never require a pasted private key in the public web demo.

Build the SaaS layer

Yes: a card/API gateway is a valuable distribution layer and can precede function creation. The customer receives an API key, a normal endpoint, and capped usage. The gateway purchases through the same paid Testril path and preserves evidence. Preserve sub-cent usage accounting and aggregate card billing; do not impose a one-cent floor on each underlying read. Begin with an allowance pilot before subscriptions.

Resolve ownership early. Recommended MVP: the gateway holds the materialization entitlement; any customer rebate is explicitly tracked service credit. A dedicated customer wallet can follow. Customer invoices and gateway cache hits do not automatically create Testril read rewards.

What to build first

Commit to **One Cent, Many Decisions, Follow the Money**, its evidence mode, and **The Second Customer Pays You**. Planning range: **11–21 active builder-days**, conditional on the small transfer-edge addition; core engine migration work is extra. Release this before waiting for function creation. Detailed function briefs cover inputs, outputs, semantics, reference fixtures, and platform gates.

Current endpoint: three functions, CDP mode, Ethereum/Arbitrum/Base Sepolia data configuration. Paid execution and payout still require rehearsal. The fifty-read example illustrates billing granularity, not a matched competitor benchmark; use bulk reads when the whole scope is known. At current rates, one 1,000-block cached read is **\$0.00102**; preparing those blocks costs **\$0.12**. Display both. Label testnet, recorded, synthetic, and sponsored runs accurately.

Recruit five developers for observed runs. Proposed continuation gate: three finish unaided, two return with a real second task, and one requests an integration or pilot. Measure wallet abandonment separately. Package each flagship as a short film, a runnable example, and an evidence-backed result.

Decisions: confirm the first audience/case; assign the transfer-edge function owner; choose rehearsal payment mode and allowance; agree gateway entitlement policy; define the observable release gate for function creation.

[Full strategy](#) · [Function briefs](#) · [Dated evidence](#)